

Sprint 2 – Search for Sprites

Student Information

Integrity Policy: All university integrity and class syllabus policies have been followed. I have neither given, nor received, nor have I tolerated others' use of unauthorized aid.

I understand and followed these policies:



Yes



No

Name: Jevin Pauly

Date: 1/31/2024

Submission Details

Final **Changelist** number: 205291

Verified build:



Yes



No

Required Configurations: DEBUG only

YouTubeLink:

<https://www.youtube.com/watch?v=9UgESripL6Q>

Discussion (What did you learn):

Learned about using Sprites, Textures

Verify Builds

- Follow the Piazza procedure on submission
 - Verify your submission compiles and works at the changelist number.
- Verify that only MINIMUM files are submitted
 - No – Generated files
 - *.pdb, *.suo, *.sdf, *.user, *.obj, *.exe, *.log, *.pdb, *.db, *.user
 - Anything that is generated by the compiler should not be included
 - No – Generated directories
 - /Debug, /Release, /Log, /ipch, /.vs
- Typical files project files that are required
 - *.sln, *.csproj, *.cs,
 - App.config, AssemblyInfo.cs, CleanMe.bat
 - Resources Directory:
 - *.tga, *.dll, *.wav, *.gls, *.azul

Standard Rules

Submit multiple times to Perforce

- Submit your work as you go to perforce several times
 - Design Patterns – no minimum
 - Sprints – minimum of 5 real submissions (generally 10+)
 - As soon as you get something working, submit to perforce
 - Have reasonable check-in comments
 - Points will be deducted if minimum is not reached

Submission Report

- Fill out the submission Report
 - No report, no grade

Code and project needs to compile and run

- Make sure that your program compiles and runs
 - Warning level 4
 - NO Warnings or ERRORS
 - Your code should be squeaky clean.
 - Code needs to work “as-is”.
 - No modifications to files or deleting files necessary to compile or run.
 - All your code must compile from perforce with no modifications.
 - Otherwise it's a 0, no exceptions

Project needs to run to completion

- If it crashes for any reason...
 - It will not be graded and you get a 0

No Containers

- Containers (No automatic containers or arrays)
- Template or generic parameters
- No arrays
 - You need to do this the old fashion way - **YOU EARNED IT**

Leave Project Settings

- Do NOT change the project or warning level
 - Any changing of level or suppression of warnings is an integrity issue

Simple C#

- No .Net
- We are using the basics
 - Types:
 - Class, Structs, intrinsic types (int, float, bool, etc...)
 - NO arrays allowed!
 - Basics language features
 - Inheritance, methods, abstract, virtual, etc...

No Debug code or files disabled

- Make sure the program has only active code
 - If you added debug code or commented out code,
 - please return to code to active state or remove it

Adding files to this project

- Make sure you add the files in the appropriate sub-directories
- Make sure any new files are successfully integrated into the project
- Make sure your new files are submitted to Perforce

Due Dates

- See Piazza for due date and time
- Submit program perforce in your student directory assignment supplied.
- Fill out your this **Submission Report** and commit to perforce
 - **ONLY** use Adobe Reader to fill out form, all others will be rejected.
 - Fill out the form and discussion for full credit.

Goals

- Sprite – abstraction
 - Image
 - Texture
- Refactor
 - Learn and understand the code
 - Adding singletons
 - Refactoring to use SLink
- Understand Texture/Image layout
 - Create several new texture/images

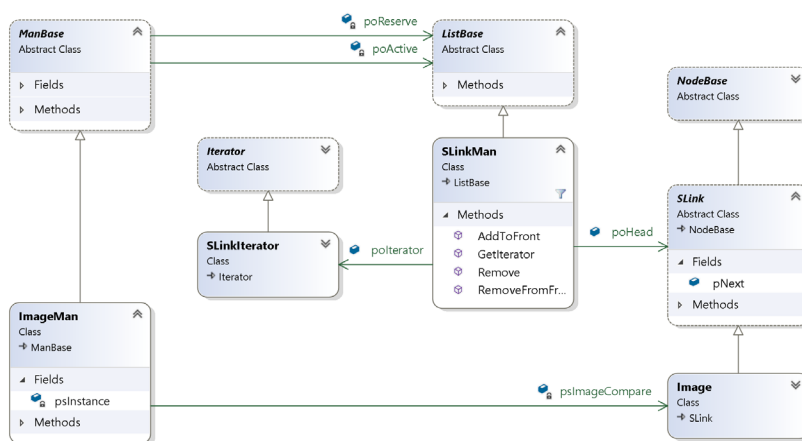
Assignments

Grading

- This sprint you are going to learn/modify the code from class.
- There are 5 steps to this refactoring

Step 1 – Refactoring 2 managers

- Refactor **TextMan** to use SLink
 - Fix the manager to use SLink
 - Since the textures are never going to be removed
 - Designed to be loaded and consistently reused in the game
 - No benefit to DLink... when SLink will work fine.
- Refactor **ImageMan** to use SLink
 - Fix the manager to use SLink



Step 2 – Add Singleton to SpriteMan

- Refactor SpriteMan to use singleton
 - Use the ImageMan as a reference
 - `SpriteMan.Create(2, 1);`
 - `SpriteMan.Destroy();`

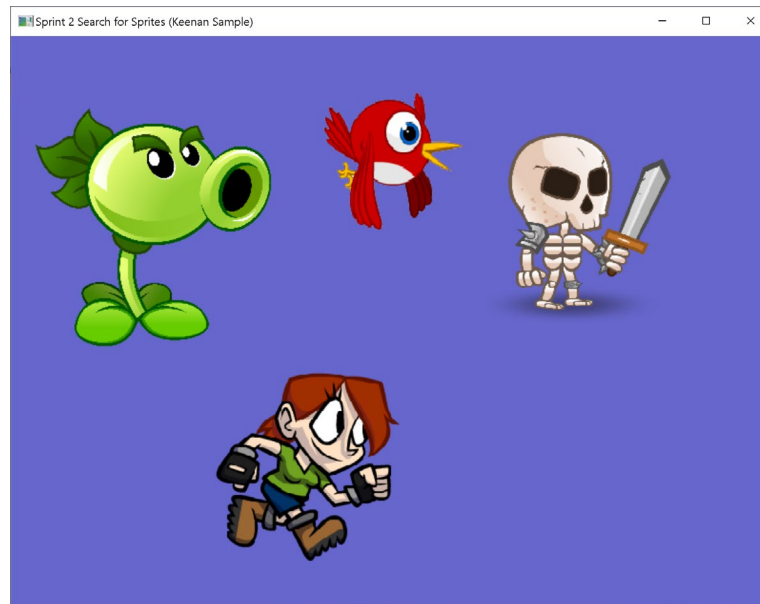
Step 3 – Remove the news in LoadContent()

- There should be no news in the LoadContent() method
 - Since pass the individual data elements directly
- Since the managers are now all Singletons there is no visible news in LoadContent()
 - `SpriteMan.Add(Sprite.Name.RedBird,
pImageRedBird,
new Azul.Rect(50, 500, 50, 50));`

Step 4 – Remove the variables in LoadContent() and globals from Game class

- LoadContent()
 - There should be no “variables” in the LoadContent() method
 - This forces the managers to Find() the resource
 - Local variables from LoadContent() such as:
 - `Image pImageRedBird`
 - `Image pImageYellowBird`
 - `Image pImageGreenBird`
 - `Image pImageWhiteBird`
- Game class
 - Remove these variables from the Game class
 - `Sprite pRedBird;`
 - `Sprite pWhiteBird;`
 - `Sprite pYellowBird;`
 - `Sprite pGreenBird;`
 - Rework the LoadContent(), Update(), Draw() to use the manager’s Find();

Step 5 – Mimic the following sample demo:



- Here is a video showing the movement:
 - <https://youtu.be/UblDLgOqExY>
 - There is motion in the video.. mimic that as well
- This demo has 4 textures with 4 images
 - Use some tool like Gimp or photo shop to get the Image Rectangles
 - PeaShooter2.tga
 - Skeleton2.tga
 - Running2.tga
 - Flying2.tga
 - Use the converted texture to load into the game
 - PeaShooter2.t azul
 - Skeleton2.t azul
 - Running2.t azul
 - Flying2.t azul
- Use a tool to measure the Rectangle Box inside the texture
 - I suggest Gimp (free) start a thread on how to find the box

Required Features

Features to show off in Demo

Show it off and talk about these required features

- Game playing mimic the sample demo
- Show the Game.cs – show off LoadContent()
- Should see the manager being used
- No visible news in LoadContent()
- Show the UML diagram of ImageMan()

Video Capture

- Have Visual Studio open and ready to go.
- Start recording
 - Show output window running
 - Speak clearly and loud
 - Discuss the features you added in this demo
 - Duration
 - Approx. 2-3 min long YouTube video capture
 - Do not exceed 5 min
 - Use OBS or similar for recording
 - High resolution / good framerate
 - Good sound quality
 - Show on the capture... it compiling and then running
 - Place YouTube link in PDF
 - Make sure it public and runs without passwords or login
 - Watch your video
 - Can you see the code clearly
 - Can you hear your voice clearly
- Don't lose points
 - Make sure everything is submitted properly
 - Don't record beyond 5 min.
 - Your audio volume was too quiet (test it and verify it)
 - Your YouTube link needs to work "As-IS" with no login
 - Don't make it private... make it unlisted
 - You check-in too many files or temporary files
 - Make sure you run CleanMe.bat before perform submission

General guidelines:

- Post on Piazza questions and clarifications
- Make sure you delete these using directives (we are not using them)
 - `using System.Collections.Generic;`
 - `using System.Linq;`
 - `using System.Text;`
 - `using System.Threading.Tasks;`

Development

- Use the project in Sprint2
- Do not copy temporary files or submit extra data
- Make sure you run CleanMe.bat before adding to perform

Submission

- Submit your SprintX directory into performce:
 - /student/<yourname>/SprintX/...
 - You need to submit a complete C# project
 - Solution, project and C# files (whatever it takes to build the project)
 - Do not submit anything that is auto generated
 - Run the supplied CleanMe.bat before submission
 - Should cleanup files
- Fill out the Submission report and submit that pdf to your student directory

Validation

Simple checklist to make sure that everything is submitted correctly

- Is the project compiling and running without any errors or warnings?
- Does the project runs **ALL** without crashing?
- Is the submission report filled in and submitted to performce?
- Follow the verification process for performce
 - Is all the code there and compiles “as-is”?
 - No extra files
- Submit the YouTube link to PDF

Hints

Most assignments will have hints in a section like this.

- Watch the lecture... look at the code supplied in class
- Study the projects in class lecture

Troubleshooting

- Print to the output window to help you debug
 - It really helps
- Ask for clarification on piazza